

CS101-Quiz6-Review

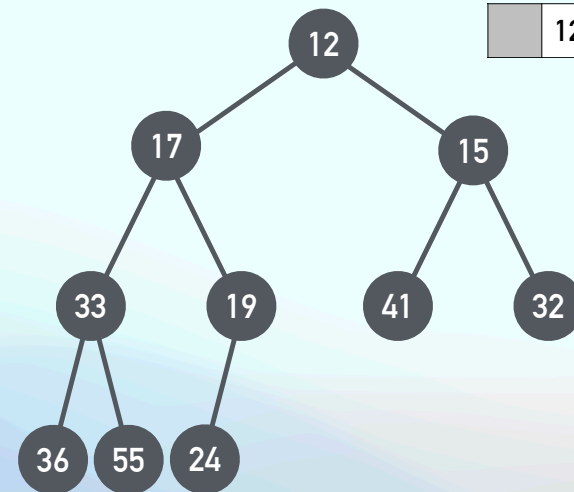
Suting Chen

CS101-Quiz6-Review

Key Points

1. Binary Heap
2. Huffman Coding
3. Binary Search Tree

Binary Heap (with complete tree)



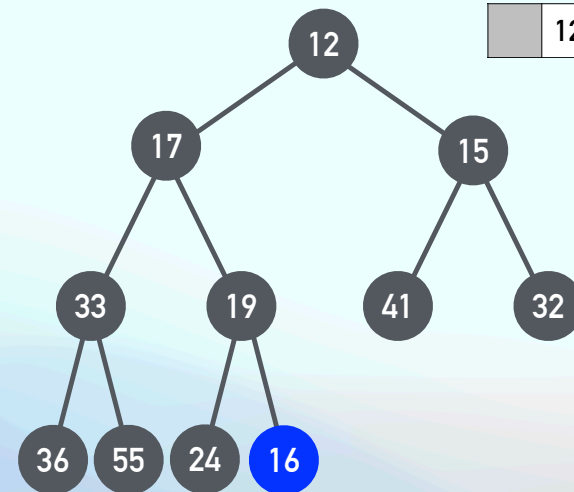
	12	17	15	33	19	41	32	36	55	24				
--	----	----	----	----	----	----	----	----	----	----	--	--	--	--

PARENT(i)
1 **return** $\lfloor i/2 \rfloor$

LEFT(i)
1 **return** $2i$

RIGHT(i)
1 **return** $2i + 1$

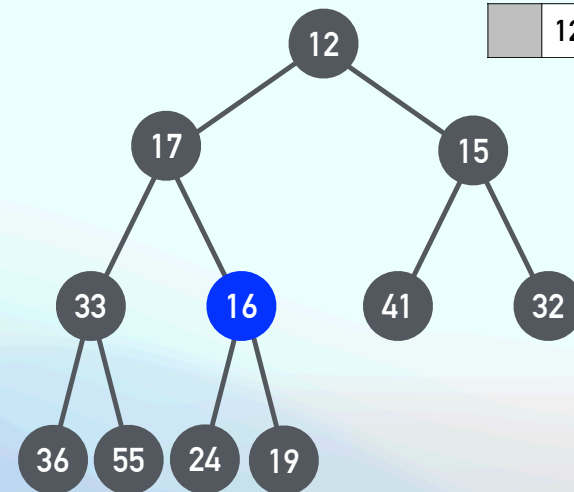
Binary Heap — push



	12	17	15	33	19	41	32	36	55	24	16			
--	----	----	----	----	----	----	----	----	----	----	----	--	--	--

1. Push the new value to the back.

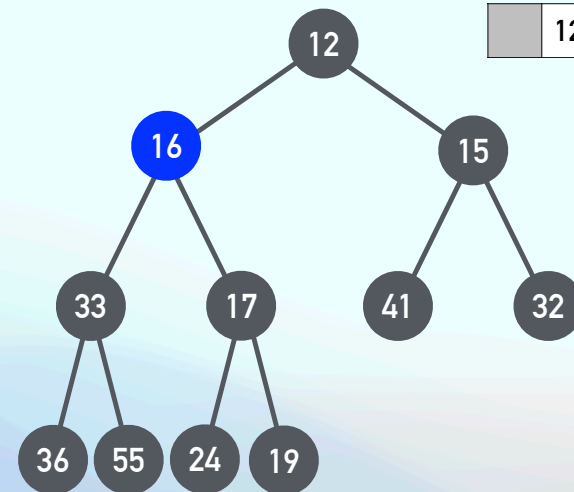
Binary Heap — push



	12	17	15	33	16	41	32	36	55	24	19			
--	----	----	----	----	----	----	----	----	----	----	----	--	--	--

1. Push the new value to the back.
2. Percolate up.

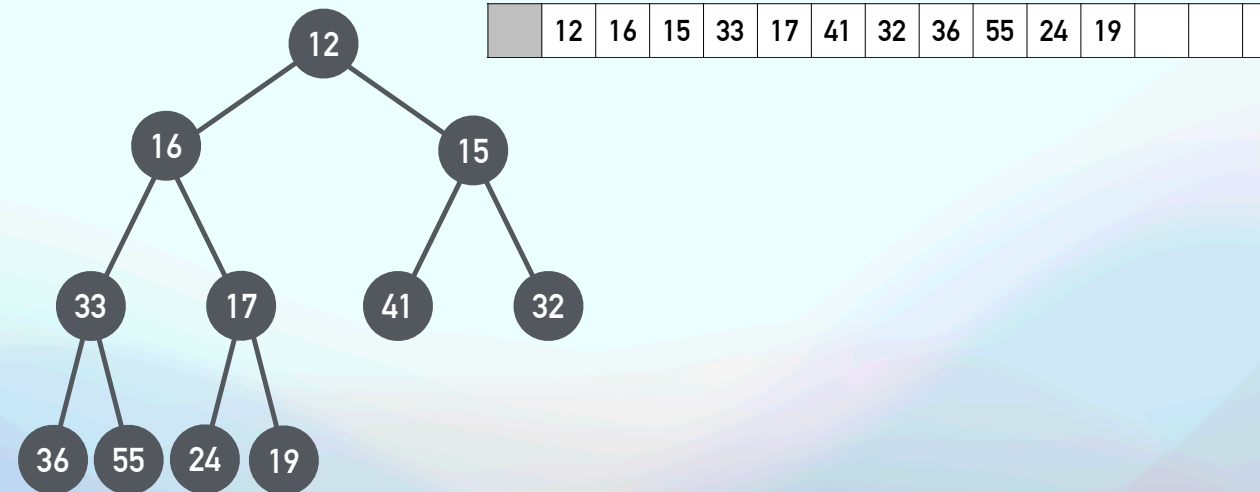
Binary Heap — push



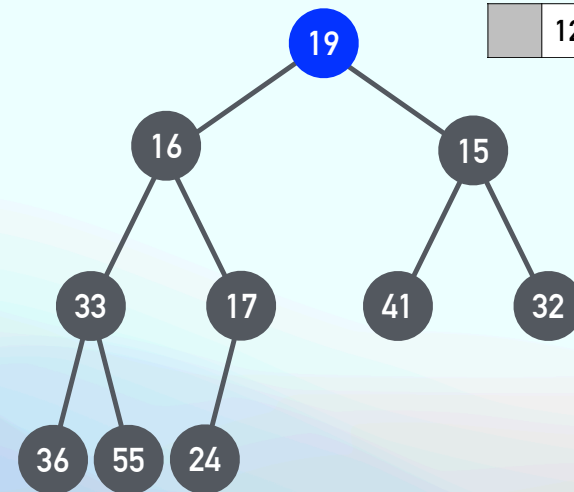
	12	16	15	33	17	41	32	36	55	24	19			
--	----	----	----	----	----	----	----	----	----	----	----	--	--	--

1. Push the new value to the back.
2. Percolate up.

Binary Heap — pop



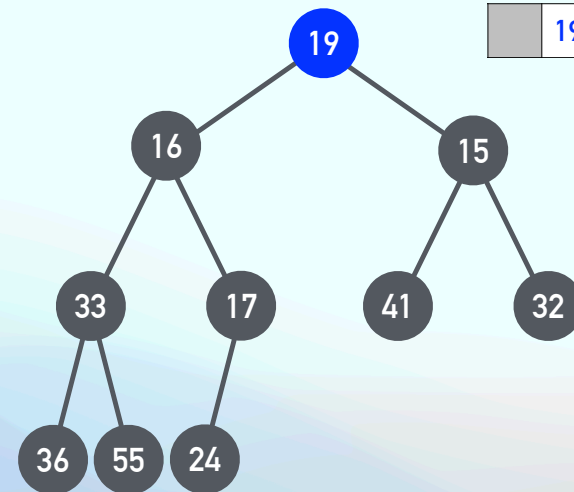
Binary Heap — pop



	12	16	15	33	17	41	32	36	55	24	19			
--	----	----	----	----	----	----	----	----	----	----	----	--	--	--

1. Copy the last entry to the top.

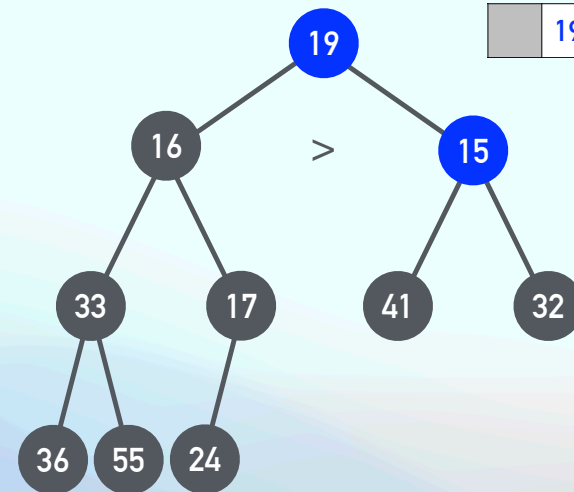
Binary Heap — pop



	19	16	15	33	17	41	32	36	55	24				
--	----	----	----	----	----	----	----	----	----	----	--	--	--	--

1. Copy the last entry to the top.

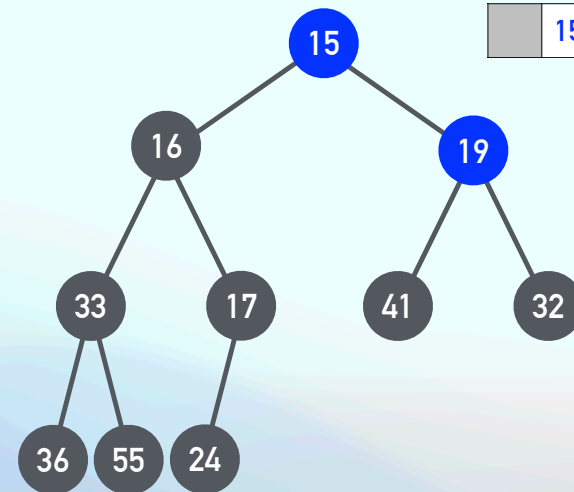
Binary Heap — pop



	19	16	15	33	17	41	32	36	55	24				
--	----	----	----	----	----	----	----	----	----	----	--	--	--	--

1. Copy the last entry to the top.
2. Percolate down.

Binary Heap — pop



	15	16	19	33	17	41	32	36	55	24				
--	----	----	----	----	----	----	----	----	----	----	--	--	--	--

1. Copy the last entry to the top.
2. Percolate down.

Binary Heap

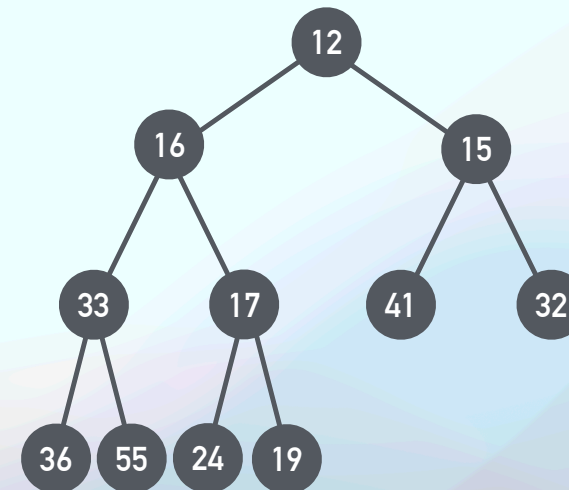
Run time analysis

top()	$\Theta(1)$
push()	$O(\log(n))$
pop()	$O(\log(n))$

Binary Heap

Run time analysis — Average Time Complexity of Push

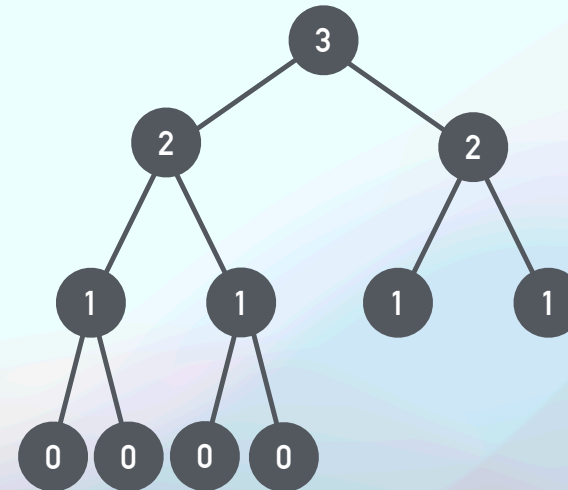
1. We push the new number at the back.
2. It would take $\Theta(h - h')$ time to place it.
 - h is the height of the tree
 - h' is the depth of where the node should be



Binary Heap

Run time analysis — Average Time Complexity of Push

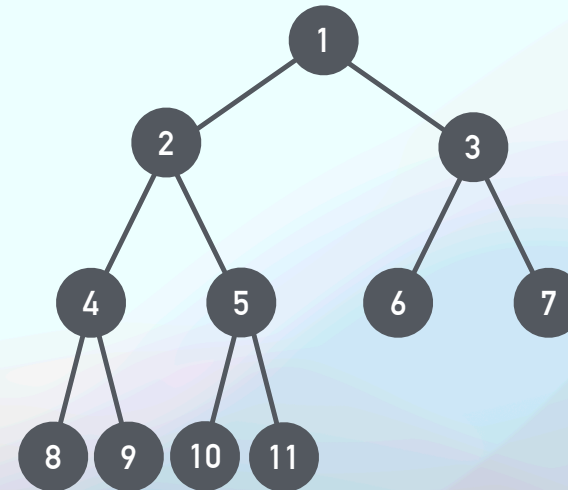
$$\begin{aligned}\frac{1}{n} \sum_{k=0}^h (h-k)2^k &= \frac{2^{h+1} - h - 2}{n} \\ &= \frac{n - h - 1}{n} \\ &= \Theta(1)\end{aligned}$$



Binary Heap

Run time analysis — Average Time Complexity of Push

1. The new number will finally “sit” in one of the positions ➡.
2. Every time we percolate up, we “eliminate” half of the possibilities.
3. The expectation would be $\Theta(1)$.

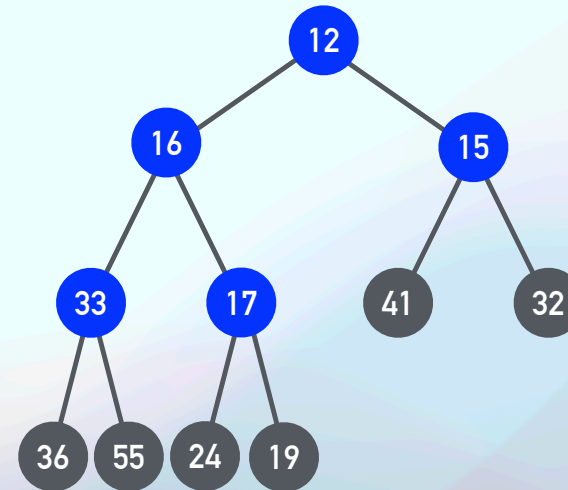


这块是一个逻辑上的简易证明，可以根据时间选择说不说

Binary Heap — Misc.

Why Floyd's Method takes $\Theta(n)$ instead of $\Theta(n \log n)$

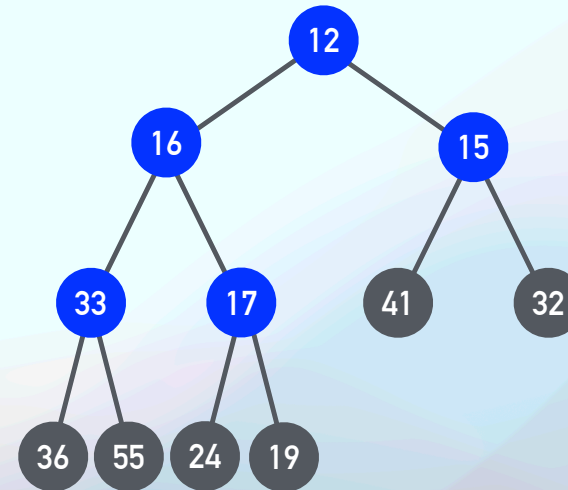
```
1 func buildHeap() {  
2   for (i = size / 2; i > 0; --i)  
3     percolateDown(i);  
4 }
```



Binary Heap — Misc.

Why Floyd's Method takes $\Theta(n)$ instead of $\Theta(n \log n)$

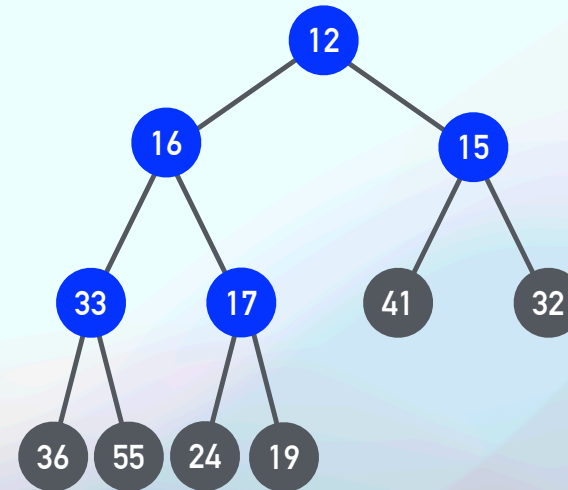
1. $\frac{1}{2}$ of the nodes go down 0 levels.
2. $\frac{1}{4}$ of the nodes go down 1 levels.
3. $\frac{1}{8}$ of the nodes go down 2 levels.



Binary Heap — Misc.

Why Floyd's Method takes $\Theta(n)$ instead of $\Theta(n \log n)$

$$\sum_{i=1}^h i \frac{n}{2^{i+2}} = \frac{n}{4} \sum_{i=1}^h \frac{i}{2^i} = \frac{n}{2}$$



错位相减，h是高度，数量级是logN

Binary Heap — Misc.

Heap Sort

1. Use Floyd's Method to build a Max-Heap.
2. Repeatedly Pop the maximum and put it at the end of the array.

- Time complexity $O(n \log n)$
- Space complexity $O(1)$

CS101-Quiz6-Review

Key Points

1. Binary Heap
- 2. Huffman Coding**
3. Binary Search Tree